



Lecture 12

Implementing Microservice Communication

Sharif University of Technology
Winter 2026



Part I

Technology Choices, RPC, REST, GraphQL & Message Brokers



Learning Objectives

Understand what makes a good inter-service communication technology

Compare RPC, REST, GraphQL, and Message Brokers

Implement basic service-to-service calls using Python

Recognize common pitfalls like tight coupling and lockstep deployments

See real-world examples from Netflix, GitHub, LinkedIn/Kafka

Why Inter-Service Communication Matters



- Microservices must talk to each other — but HOW matters enormously
- Poor choices create tight coupling, breaking changes, and deployment pain
- The wrong technology can make your system brittle and hard to evolve
- Key insight: your communication style should drive your technology choice, not the reverse
 - Example: if you need async fire-and-forget → use a message broker, not REST
 - Example: if you need low-latency typed RPC → consider gRPC over HTTP
- This chapter surveys the most popular options and their real-world trade-offs

What We Want From Communication Technology



Backward Compatibility

Adding new fields shouldn't break existing consumers



Technology Agnostic

Don't lock consumers to a specific platform or language



Hide Implementation

Internal details must not leak through the interface



Explicit Interface

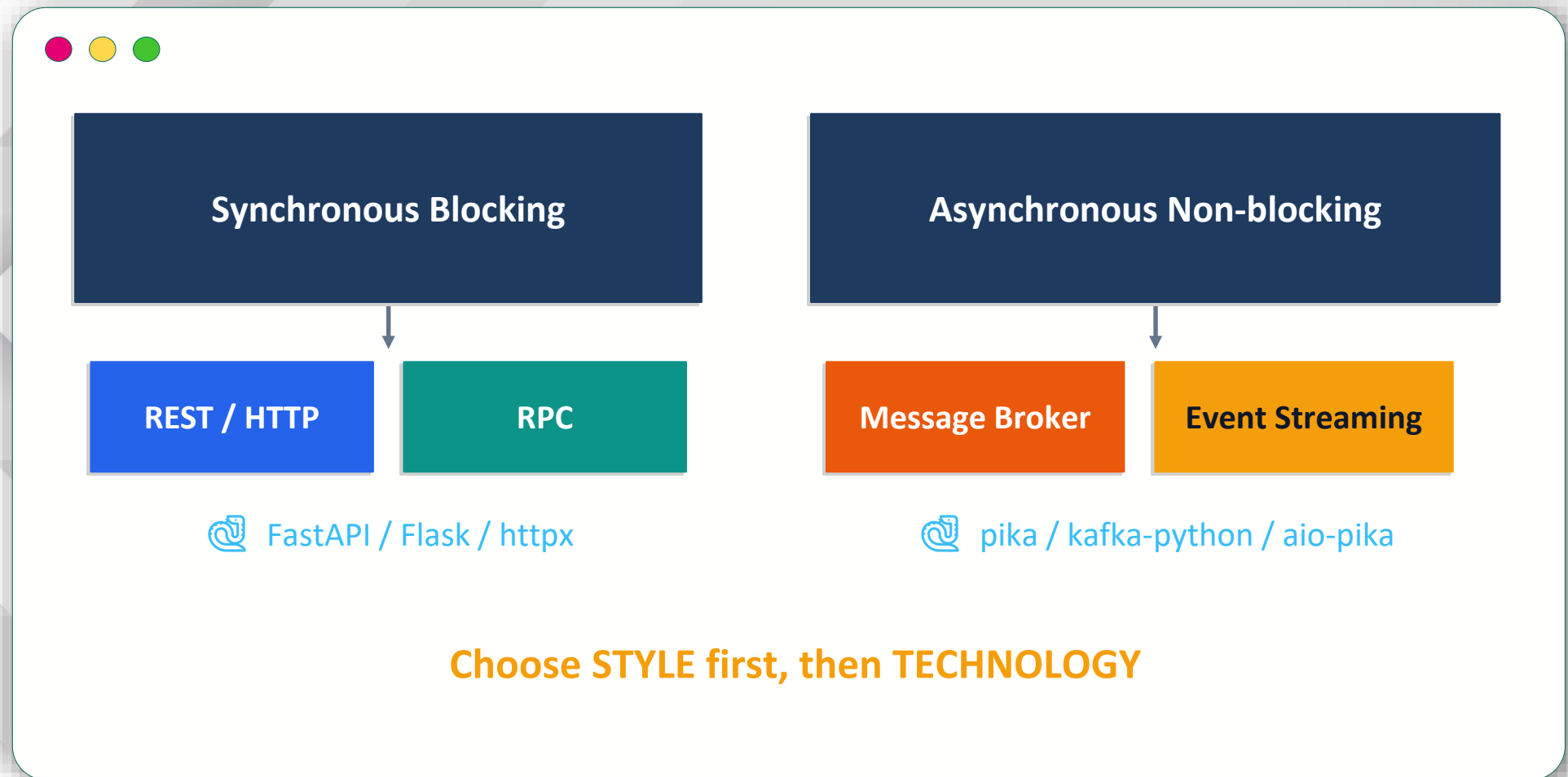
Both sides must be clear about what is exposed



Simple for Consumers

Easy to integrate without excessive boilerplate

Technology Landscape: Pick Your Style First





Remote Procedure Calls (RPC)

Making remote calls look local — the good, the bad, and the ugly

What is RPC? The Core Concept



- RPC = Remote Procedure Call: make a call to a function that runs on another machine
- Goal:
 - make **remote** calls **look** and **feel** like **local function** calls
- Uses an Interface Definition Language (IDL) to describe the contract
- Common implementations:
 - SOAP (WSDL)
 - XML-based, legacy, very heavyweight for developers
 - gRPC (Protocol Buffers)
 - : Modern, uses HTTP/2, binary protocol buffers, excellent performance
 - Java RMI
 - JVM-only, tight coupling, largely avoided today
- Key mechanism: client and server stubs generated from the schema

gRPC in Python — Define the Proto Schema

```
1 # customer.proto
2 syntax = "proto3";
3
4 package customer;
5
6 service CustomerService {
7     rpc GetCustomer (GetCustomerRequest) returns (Customer);
8     rpc CreateCustomer (CreateCustomerRequest) returns (Customer);
9 }
10
11 message GetCustomerRequest {
12     string id = 1;
13 }
14
15 message Customer {
16     string id = 1;
17     string first_name = 2;
18     string last_name = 3;
19     string email = 4;
20     // field 5 reserved for future use
21 }
22
23 message CreateCustomerRequest {
24     string first_name = 1;
25     string last_name = 2;
26     string email = 3;
27 }
```

- Use the .proto file to define messages and RPC methods in a language-neutral way
- Use protoc (Protocol Buffer compiler) to generate client/server code and data models automatically
- At runtime, gRPC uses the generated code to serialize requests/responses efficiently and enable communication between services

💡 Run: `python -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=. customer.proto`

gRPC in Python — Server Implementation

```
1 import grpc
2 from concurrent import futures
3 import customer_pb2, customer_pb2_grpc
4
5 class CustomerServicer(customer_pb2_grpc.CustomerServiceServicer):
6
7     def GetCustomer(self, request, context):
8         # Fetch from database
9         customer = db.get_customer(request.id)
10        if not customer:
11            context.set_code(grpc.StatusCode.NOT_FOUND)
12            context.set_details("Customer not found")
13            return customer_pb2.Customer()
14        return customer_pb2.Customer(
15            id=customer.id,
16            first_name=customer.first_name,
17            last_name=customer.last_name,
18            email=customer.email,
19        )
20
21 def serve():
22     server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
23     customer_pb2_grpc.add_CustomerServiceServicer_to_server(
24         CustomerServicer(), server
25     )
26     server.add_insecure_port("[::]:50051")
27     server.start()
28     server.wait_for_termination()
```

💡 Binary protocol buffers → much faster than JSON over the wire

gRPC in Python — Client Implementation

```
1 import grpc
2 import customer_pb2, customer_pb2_grpc
3
4 def get_customer(customer_id: str):
5     with grpc.insecure_channel("localhost:50051") as channel:
6         stub = customer_pb2_grpc.CustomerServiceStub(channel)
7         try:
8             response = stub.GetCustomer(
9                 customer_pb2.GetCustomerRequest(id=customer_id),
10                 timeout=5.0 # Always set a timeout!
11             )
12             return {
13                 "id": response.id,
14                 "name": f"{response.first_name} {response.last_name}",
15                 "email": response.email,
16             }
17         except grpc.RpcError as e:
18             if e.code() == grpc.StatusCode.NOT_FOUND:
19                 return None
20             raise # Re-raise unexpected errors
21
22 customer = get_customer("cust-123")
23 print(customer)
```

💡 Always handle gRPC errors explicitly — treat it like network I/O, not a local call!

RPC Pitfalls — What Can Go Wrong



Remote $\neq$ Local

Marshaling cost, network latency, and partial failure don't exist locally. Always handle timeouts.

Technology Coupling

Java RMI ties both client AND server to the JVM. gRPC/Thrift are multi-language — prefer them.

Brittleness (Binary Stubs)

Removing a field from a shared type $\rightarrow$ all consumers break. Binary types become 'expand only'.

The Fallacies of Distributed Computing

"The network is reliable" is Fallacy #1. Plan for failures, timeouts, and retries from day one.

RPC Brittleness



The server-side Customer object evolves over time...



```
1  # Version 1: works fine
2  class Customer:
3      def __init__(self, id, first_name, last_name, email, age):
4          self.id = id
5          self.first_name = first_name
6          self.last_name = last_name
7          self.email = email
8          self.age = age # Nobody uses this field!
9
10 # Version 2: Remove 'age' (nobody uses it, right?)
11 class Customer:
12     def __init__(self, id, first_name, last_name, email):
13
```

✗ PROBLEM:

- Old clients using binary serialization
- WILL BREAK when deserializing because 'age' is missing!

☑ With Protocol Buffers (gRPC):

- Field NUMBERS are stable; field names can even change safely

RPC: When to Use It (and When to Avoid)



- ☑ GOOD: You control both client and server (internal microservices)
- ☑ GOOD: Performance-critical paths — gRPC binary is ~7x faster than JSON/REST
- ☑ GOOD: Strongly-typed contracts needed across multiple teams
- ☑ GOOD: Streaming scenarios (gRPC over HTTP/2 supports bidirectional streams)
- ✗ AVOID: Public APIs where many external clients need access
- ✗ AVOID: When technology lock-in is a concern (Java RMI, SOAP)
- ✗ AVOID: When you cannot enforce coordinated schema upgrades
- 📌 **Recommendation: gRPC is the modern, preferred RPC choice**



REST over HTTP

The industry default — practical, flexible, and everywhere

REST: Core Concepts



- REST = **Representational State Transfer** — an **architectural style**, not a **protocol**
- Core concept:
 - **RESOURCES**: Things the service knows about (Customer, Order...)
 - Resources exposed at stable URIs; operated on with standard HTTP verbs
 - **GET** /customers/123 → retrieve (idempotent, safe)
 - **POST** /customers → create new customer
 - **PUT** /customers/123 → update (idempotent)
 - **DELETE** /customers/123 → remove
 - **Representations** are decoupled from storage — JSON, XML, or any format
- HTTP gives us: caching, load balancing, auth, monitoring out of the box
- **JSON** is the **default serialization** format (replaced XML as the de facto standard)
- **REST over HTTP** is the **sensible default for synchronous request-response APIs**

REST in Python — FastAPI Customer Service

```
1 from fastapi import FastAPI, HTTPException, status
2 from pydantic import BaseModel
3 from typing import Optional
4
5 app = FastAPI(title="Customer Service", version="1.0.0")
6
7 class Customer(BaseModel):
8     id: str
9     first_name: str
10    last_name: str
11    email: str
12    # Note: adding new OPTIONAL fields is backward-compatible
13    phone: Optional[str] = None # Added in v1.1 - safe!
14
15 class CreateCustomerRequest(BaseModel):
16     first_name: str
17     last_name: str
18     email: str
19
20 @app.get("/customers/{customer_id}", response_model=Customer)
21 async def get_customer(customer_id: str):
22     customer = await db.get_customer(customer_id)
23     if not customer:
24         raise HTTPException(status_code=404, detail="Customer not found")
25     return customer
26
27 @app.post("/customers", response_model=Customer, status_code=status.HTTP_201_CREATED)
28 async def create_customer(request: CreateCustomerRequest):
29     return await db.create_customer(request)
```

💡 FastAPI auto-generates OpenAPI docs at /docs — **explicit schema for free!**



REST API Documentation Tool | Swagger UI

The screenshot displays the Swagger UI for the Swagger Petstore API. The interface includes a top navigation bar with the SwaggerHub logo and user information (frank.kilcommins). Below the navigation bar, the API is identified as 'PetStoreSwagger' version '1.0.0'. The main content area is titled 'Swagger Petstore' and includes a description: 'This is a sample Petstore server. You can find out more about Swagger at <http://swagger.io> or on [#swagger](irc://irc.freenode.net).' It also provides links for 'Terms of service', 'Contact the developer', 'Apache 2.0', and 'Find out more about Swagger'. A 'Servers' section shows the selected server as 'https://petstore.swagger.io/v2' with an 'Authorize' button. The 'pet' resource is listed with the description 'Everything about your Pets'. The API endpoints are listed in a table:

Method	Endpoint	Description	Expand	Lock	Copy
POST	/pet	Add a new pet to the store	▼	🔒	📋
PUT	/pet	Update an existing pet	▼	🔒	📋
GET	/pet/findByStatus	Finds Pets by status	▼	🔒	📋
GET	/pet/findByTags	Finds Pets by tags	▼	🔒	📋
GET	/pet/{petId}	Find pet by ID	▼	🔒	📋
POST	/pet/{petId}	Updates a pet in the store with form data	▼	🔒	📋
DELETE	/pet/{petId}	Deletes a pet	▼	🔒	📋

REST in Python — Consuming a Service

```
1 import httpx
2 from typing import Optional
3 import logging
4
5 logger = logging.getLogger(__name__)
6
7 class CustomerClient:
8     def __init__(self, base_url: str, timeout: float = 5.0):
9         self.base_url = base_url
10        self.timeout = timeout
11
12    async def get_customer(self, customer_id: str) -> Optional[dict]:
13        async with httpx.AsyncClient(timeout=self.timeout) as client:
14            try:
15                response = await client.get(
16                    f"{self.base_url}/customers/{customer_id}"
17                )
18                if response.status_code == 404:
19                    return None
20                response.raise_for_status()
21                data = response.json()
22                return {
23                    "id": data["id"],
24                    "name": f"{data['first_name']} {data['last_name']}",
25                    "email": data["email"],
26                }
27            except httpx.TimeoutException:
28                logger.error("Timeout calling customer service")
29                raise
```

💡 Tolerant Reader pattern: only extract what you **need**, ignore the rest

HTTP Verbs & Status Codes



GET	<code>/customers/{id}</code>	Read a resource — IDEMPOTENT, safe to cache	200 OK / 404
POST	<code>/customers</code>	Create a new resource — NOT idempotent	201 Created
PUT	<code>/customers/{id}</code>	Replace entire resource — IDEMPOTENT	200 / 204
PATCH	<code>/customers/{id}</code>	Partial update — fields provided only	200 / 204
DELETE	<code>/customers/{id}</code>	Remove resource — IDEMPOTENT	204 No Content

HATEOAS — Hypermedia as the Engine of Application State

Example Response

```
{
  "album": "Give Blood",
  "artist": "The Brakes",
  "_links": {
    "self": { "href": "/albums/1" },
    "artist": {
      "href": "/artists/theBrakes"
    },
    "purchase": {
      "href": "/purchases/1234"
    }
  }
}
```

The Idea

- Clients follow links, not hardcoded URLs
- URLs can change without breaking clients
- API is self-describing and discoverable
- Like browsing the web — follow links

Reality Check

- Rarely practiced in real systems
- Added complexity for limited gain
- Most teams skip it and do fine

REST Challenges — Know the Trade-offs



- Performance: JSON payloads larger than binary; TCP overhead per request
- No built-in streaming (unlike gRPC) — workarounds exist (SSE, WebSockets)
- HATEOAS chatty protocols — multiple round trips to navigate links
- Client code generation historically manual — OpenAPI/Swagger now helps greatly
- HTTP/3 (QUIC) is coming — reduces TCP connection overhead significantly
- Not ideal for general microservice-to-microservice comms when performance matters
- ☒ Still the right **default** for: **external APIs**, **browser clients**, varied consumers



GraphQL

Client-defined queries — solving the over-fetching problem

GraphQL — The Problem It Solves



- Classic REST problem: mobile app needs 5 fields from a 40-field Customer response
 - Over-fetching: receiving data you don't need (wastes mobile bandwidth)
 - Under-fetching: making multiple round-trips to assemble a screen's worth of data
- GraphQL solution: client defines **EXACTLY** what it wants in a **single** query
 - Single endpoint (usually POST /graphql) replaces many REST endpoints
 - Query language lets clients ask for nested resources in one call
- Real-world adoption: GitHub moved its v4 API to GraphQL; Facebook invented it
- Best fit: API gateway for external clients / mobile apps — NOT internal microservices

GraphQL Type System

```
● ● ●  
  
# Scalar types (built-in)  
# String Int Float Boolean ID  
  
type Product {  
  id: ID!  
  name: String!  
  price: Float  
  inStock: Boolean!  
  tags: [String]  
}  
  
type Query {  
  product(id: ID!): Product  
  products: [Product!]!  
}
```

◆ Types & Fields

Every type is a named object with typed fields. The schema is the contract between client and server.

! Non-Null (!)

A ! suffix means the field is guaranteed non-null. [Type!]! means a non-null list of non-null items.

🔑 The Query Type

Query is the entry point — like REST routes. Every field on Query is a callable operation.

📦 Scalars

Leaf nodes: String, Int, Float, Boolean, ID. Custom scalars (e.g. Date, UUID) can be defined too.

Anatomy of a GraphQL Query

```
query CustomerPage($id: ID!) {  
  customer(id: $id) {  
    name  
    email  
    recentOrders {  
      product  
      value  
      status  
    }  
  }  
}
```

①

Operation & Variable

query keyword + named operation + typed variable (\$id: ID!). Use mutation for writes.

②

Field with Argument

Pass the variable as an argument. Arguments filter or parameterise the resolver.

③

Scalar Field (leaf)

name and email return primitive values — these are the leaves; no sub-selection needed.

④

Nested Object Field

recentOrders returns a list of objects, so it needs its own selection set in { }.

GraphQL in Python — Strawberry Framework

```
1 import strawberry
2 from strawberry.fastapi import GraphQLRouter
3 from fastapi import FastAPI
4 from typing import List
5
6 @strawberry.type
7 class Order:
8     id: str
9     product: str
10    value: float
11    status: str
12
13 @strawberry.type
14 class Customer:
15     id: str
16     name: str
17     email: str
18
19     @strawberry.field
20     async def recent_orders(self) -> List[Order]:
21         orders = await order_service.get_by_customer(self.id, limit=5)
22         return [Order(**o) for o in orders]
23
24 @strawberry.type
25 class Query:
26     @strawberry.field
27     async def customer(self, id: str) -> Customer:
28         data = await customer_service.get(id)
29         return Customer(**data)
30
31 schema = strawberry.Schema(Query)
32 app = FastAPI()
33 app.include_router(GraphQLRouter(schema), prefix="/graphql")
```



Client query: { customer(id:"123") { name
email recentOrders { product value status } } }

GraphQL In Python — Client Query vs REST

```
1 import httpx
2 # REST approach
3 async def get_customer_page_rest(customer_id: str):
4     async with httpx.AsyncClient() as client:
5         cust = await client.get(f"http://customers/{customer_id}")
6         orders = await client.get(f"http://orders?customer={customer_id}&limit=5")
7         return {"customer": cust.json(), "orders": orders.json()}
8
9 # GraphQL approach
10 QUERY = """
11 query CustomerPage($id: ID!) {
12   customer(id: $id) {
13     name
14     email
15     recentOrders {
16       product
17       value
18       status
19     }
20   }
21 }
22 """
23 async def get_customer_page_graphql(customer_id: str):
24     async with httpx.AsyncClient() as client:
25         result = await client.post(
26             "http://api-gateway/graphql",
27             json={"query": QUERY, "variables": {"id": customer_id}}
28         )
29     return result.json()["data"]
```

💡 GraphQL reduces round-trips and bandwidth — especially valuable on mobile networks

GraphiQL — The Built-in Explorer

```
GraphiQL - localhost:8000/graphql
```

Execute Query Docs History

```
query GetProduct {  
  product(id: "42") {  
    name  
    price  
    inStock  
  }  
}
```

```
{  
  "data": {  
    "product": {  
      "name": "Widget Pro"  
      "price": 29.99  
      "inStock": true  
    }  
  }  
}
```



Schema Explorer

Click "Docs" to browse every type, field, and argument — auto-generated from your schema.



Autocomplete

Press Ctrl+Space mid-query to see available fields for the current type.



Run & Inspect

Hit ▶ Execute. Response appears instantly in the right pane as formatted JSON.



Enable in FastAPI

`add_graphql_route(app, schema, graphiql=True).`
Visit `/graphql` in browser.

GraphQL: Strengths vs. Challenges



Where GraphQL Shines

- Single endpoint for all client needs
- Client specifies exactly what it needs — no over/under-fetching
- Reduces round trips on mobile / slow networks
- Auto-generated docs and schema explorer
- GitHub, Shopify, Twitter all use it externally
- Excellent for varied UIs (mobile, web, TV) hitting same backend

Known Challenges

- Complex queries can hammer servers (N+1 problem)
- Caching is much harder than REST (no HTTP cache headers)
- Mostly read-oriented — mutations less natural
- Feels like data layer, can blur service boundaries
- Not for service-to-service communication
- Extra server-side complexity with resolvers



Message Brokers

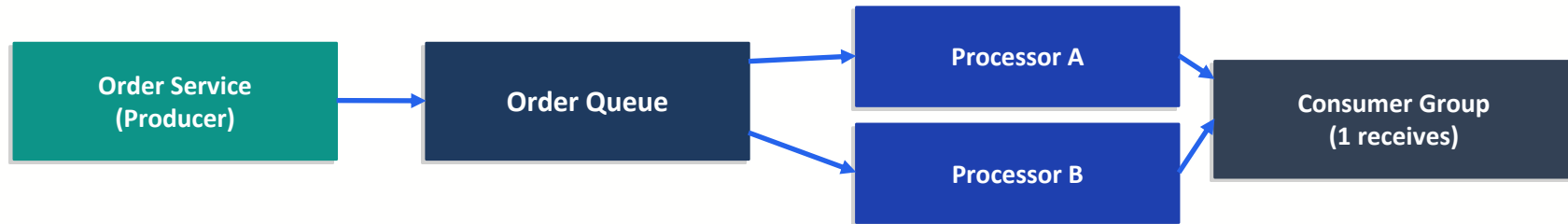
Asynchronous communication — decoupling producers from consumers

Why Use a Message Broker?

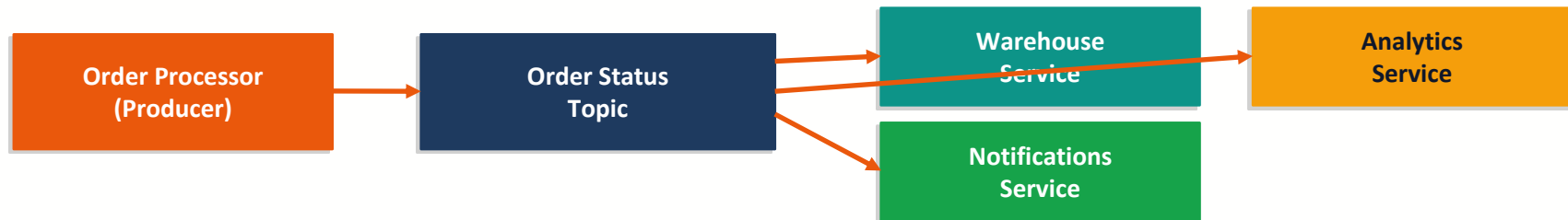


- Asynchronous communication: sender doesn't wait for receiver to respond
- GUARANTEED DELIVERY: broker holds message until consumer is ready
 - Compare to REST: if downstream is down → your HTTP call fails
 - With broker: if downstream is down → message waits in queue safely
- Decoupling: producer doesn't know (or care) who the consumers are
- Key property: Durability — messages persisted to disk, survive broker restarts
- Consumer groups: multiple instances compete for messages → natural load balancing
- Real-world: Kafka powers LinkedIn, Netflix, Twitter at massive scale

Queues vs Topics — Key Architecture Difference



QUEUE: load-balanced, one consumer gets each message



TOPIC: broadcast, EACH subscriber gets a copy of every message

Topics → event-driven | Queues → request/response async

Message Broker in Python — RabbitMQ with pika

```
1 import pika, json
2
3 # — PRODUCER (publishes messages) —————
4 def publish_order(order: dict):
5     connection = pika.BlockingConnection(
6         pika.ConnectionParameters(host="localhost")
7     )
8     channel = connection.channel()
9     channel.queue_declare(queue="orders", durable=True)
10    channel.basic_publish(
11        exchange="",
12        routing_key="orders",
13        body=json.dumps(order),
14        properties=pika.BasicProperties(
15            delivery_mode=2,
16            content_type="application/json",
17        )
18    )
19    connection.close()
20
21 # — CONSUMER (processes messages) —————
22 def on_order_received(ch, method, properties, body):
23     order = json.loads(body)
24     print(f"Processing order: {order['id']}")
25     process_order(order)
26     ch.basic_ack(delivery_tag=method.delivery_tag) # MUST ack!
27
28 channel.basic_qos(prefetch_count=1)
29 channel.basic_consume("orders", on_order_received)
30 channel.start_consuming()
```

💡 durable=True + delivery_mode=2 =
guaranteed delivery; always ACK!

Apache Kafka — When You Need Scale



- Kafka is NOT just a message broker — it's a distributed event log
- Messages are RETAINED for a configurable period (even forever!)
 - Consumers can rewind and reprocess old messages
 - Newly deployed services can catch up on past events
- LinkedIn built Kafka to replace dozens of separate message clusters
 - 50,000+ producers and consumers on a single cluster is possible
- KSQL: SQL-like queries over live Kafka topics — like a live database view
- Ordering guaranteed WITHIN a partition (not globally across partitions)
- Python: kafka-python or confluent-kafka libraries

Apache Kafka in Python — Producer & Consumer

```
1 from kafka import KafkaProducer, KafkaConsumer
2 import json
3
4 # — PRODUCER —
5 producer = KafkaProducer(
6     bootstrap_servers=["localhost:9092"],
7     value_serializer=lambda v: json.dumps(v).encode("utf-8"),
8     acks="all",          # wait for all replicas - maximum durability
9     retries=5,
10 )
11 def emit_order_paid_event(order_id: str, amount: float):
12     event = {
13         "event_type": "OrderPaid",
14         "order_id": order_id,
15         "amount": amount,
16     }
17     producer.send("order-status", value=event, key=order_id.encode())
18     producer.flush()
19
20 # — CONSUMER (e.g., Warehouse Service) —
21 consumer = KafkaConsumer(
22     "order-status",
23     bootstrap_servers=["localhost:9092"],
24     group_id="warehouse-service",    # consumer group = competing consumers
25     auto_offset_reset="earliest",    # start from beginning if new group
26     value_deserializer=lambda m: json.loads(m.decode("utf-8")),
27 )
28 for message in consumer:
29     handle_order_paid(message.value)
```

💡 `group_id` ensures competing consumers;
each partition only goes to one group
member

Broker Comparison: RabbitMQ vs Kafka vs Cloud

Feature	RabbitMQ	Apache Kafka	AWS SQS/SNS
Message Retention	Until consumed	Configurable (forever)	4-14 days
Ordering	Per queue	Per partition	FIFO queue (optional)
Throughput	High	Very high	High (managed)
Replay	✗ No	☑ Yes	✗ No
Best For	Task queues, RPC async	Event streaming, audit log	Simple cloud-native queuing
Python Library	pika / aio-pika	kafka-python / confluent	boto3

Key Takeaways



✓ Communication style first, then technology

✓ Message brokers for async & decoupling

✓ gRPC: best for internal high-performance RPC

✓ Kafka for event streaming at scale

✓ REST: default for external/synchronous APIs

✓ Always set timeouts on remote calls

✓ GraphQL: external mobile/API aggregation

✓ Tolerant Reader: ignore what you don't need



Part II

Schemas, Versioning & Service Discovery



Learning Objectives

Serialization Formats

Structural vs Semantic contract breakage and how schemas help

Handling Change Between Microservices

Understand service discovery: DNS, Kubernetes

Service mesh and API gateway strategy for our architecture



Serialization Formats

How your data travels over the wire

Serialization Formats — Text vs Binary



JSON (Text)

✓ Human readable, universal browser support, simple

⚠ Larger payload, no schema by default, slow parse

XML (Text)

✓ XPath/XSLT tooling, namespaces, mature ecosystem

⚠ Very verbose, complex parsing, largely replaced by JSON

Avro (Text/Binary)

✓ Schema in payload, great for Kafka, dynamic typing support

⚠ Less common outside Hadoop/Kafka ecosystem

Protocol Buffers (Binary)

✓ Compact, fast, field numbers for compatibility, gRPC default

⚠ Not human readable, requires .proto file

Serialization Formats — Comparison



Format	Type	Schema?	Size	Speed	Human Readable?	Use Case
JSON	Text	Optional	Medium	Medium	Yes	REST APIs, web
XML	Text	Optional	Large	Slow	Yes	SOAP, legacy
Protobuf	Binary	Required	Small	Fast	No	gRPC, internal
Avro	Binary	Built-in	Small	Fast	No	Kafka payloads
Thrift	Binary	Required	Small	Fast	No	Multi-language RPC
MessagePack	Binary	Optional	Small	Fast	No	High-perf REST

Schemas & Contract Safety

Making the implicit explicit — catching breaks before production

Two Types of Contract Breakage



Structural Breakage

Structure of interface changes

- Removing a required field
- Removing a method/endpoint
- Adding a new required field
- Renaming a field

Detected by: Schema comparison tools

Tools: Protolock, openapi-diff, json-schema-diff-validator

Easier to catch automatically!

Semantic Breakage

Structure stays the same; BEHAVIOR changes

- calculate(a,b) adds instead of multiplies
- A field's meaning silently changes
- Side effects change unexpectedly

Detected by: Tests ONLY

Tools: Pact (consumer-driven contract tests)

Harder — you must write tests!

The Schema Debate



If you are consuming data from a schemaless API, you still have expectations about what data should be there. You DO have a schema — it's just implicit rather than explicit.

— Building Microservices, Sam Newman

Python Schemas with Pydantic — Structural Safety

```
1 from pydantic import BaseModel, EmailStr, validator
2 from typing import Optional
3 from datetime import date
4
5 # — SCHEMA DEFINITION —————
6 class CustomerV1(BaseModel):
7     id: str
8     first_name: str
9     last_name: str
10    email: EmailStr
11    # Backward-compatible addition:
12    date_of_birth: Optional[date] = None # Optional = safe expansion
13
14    class Config:
15        extra = "ignore" # Tolerant Reader: ignore unknown fields!
16
17 # — STRUCTURAL VALIDATION —————
18 try:
19     customer = CustomerV1(
20         id="cust-1",
21         first_name="Alice",
22         last_name="Smith",
23         email="alice@example.com",
24         unknown_future_field="xyz" # ignored due to extra="ignore"
25     )
26     print(customer.json())
27 except ValidationError as e:
28     print(f"Contract broken: {e}") # Caught immediately!
29
30 # — TESTING BACKWARD COMPAT (in CI) —————
31 def test_response_still_valid(api_response: dict):
32     CustomerV1(**api_response) # Raises if structure changed!
```

- Config customizes Pydantic model behavior.
- `extra = "ignore"` lets unknown fields pass silently.
- Ensures backward/forward API compatibility

OpenAPI Schema — Auto-Generated from FastAPI

```
1  from fastapi import FastAPI
2  from pydantic import BaseModel
3  from typing import Optional
4
5  app = FastAPI(
6      title="Customer Service API",
7      version="1.2.0",
8      description="Manages customer records"
9  )
10
11  class Customer(BaseModel):
12      """Customer resource - v1 schema."""
13      id: str
14      first_name: str
15      last_name: str
16      email: str
17      phone: Optional[str] = None
18
19  @app.get(
20      "/customers/{customer_id}",
21      response_model=Customer,
22      summary="Get a customer by ID",
23      responses={404: {"description": "Customer not found"}}
24  )
25  async def get_customer(customer_id: str):
26      """Returns the customer with the given ID."""
27      ...
```

- **FastAPI auto-generates the OpenAPI spec:**

- GET /openapi.json → full machine-readable schema
- GET /docs → Swagger UI
- GET /redoc → ReDoc UI
- CI: use openapi-diff to compare old vs new spec and **FAIL** on breaks!

Catching Breaking Changes in CI



Goal: fail the CI build BEFORE the breaking change reaches production!



Protolock

For: Protocol Buffers (gRPC)

protolock commit && protolock status — fails CI on breaking changes



openapi-diff

For: REST / OpenAPI schemas

Compare old and new openapi.json; fails if incompatible change detected



json-schema-diff-validator

For: JSON Schema

npm tool; compares two JSON Schema files for backward compatibility



Pact

For: Consumer-driven contract tests

Consumers publish pacts; provider verifies against them in CI



Handling Change Between Microservices

The #1 practical challenge in microservice architectures

Five Strategies to Avoid Breaking Changes



- Expansion Changes — add new things, never remove old things
 - Safe: add optional field
 - Unsafe: remove field, rename field
- Tolerant Reader — only extract what YOU need, ignore the rest
- Right Technology — prefer tech that makes evolution easy
 - Protocol Buffers field numbers
 - Avro schema evolution
- Explicit Interface — use schemas so both sides know the contract
 - FastAPI OpenAPI
 - gRPC .proto files
- Catch Breaks Early — schema diff tools in your CI pipeline
 - Never discover a breaking change in production!

Expansion Changes — Safe vs Unsafe Examples

```
1  from pydantic import BaseModel
2  from typing import Optional
3
4  # — VERSION 1 (current) —————
5  class CustomerV1(BaseModel):
6      id: str
7      first_name: str
8      last_name: str
9      email: str
10
11 # — VERSION 1.1 — SAFE EXPANSION —————
12 class CustomerV1_1(BaseModel):
13     id: str
14     first_name: str
15     last_name: str
16     email: str
17     phone: Optional[str] = None # ✅ Optional = safe
18     date_of_birth: Optional[str] = None # ✅ Optional = safe
19
20 # — VERSION 2.0 — BREAKING CHANGES —————
21 class CustomerV2_BREAKING(BaseModel):
22     id: str
23     full_name: str # ❌ BREAKING: removed first_name + last_name
24     email: str
```

💡 Think of response types as "expand-only" — you can **ADD** fields, but you should **NEVER** remove or rename without a **versioning plan**

Semantic Versioning for Microservices



2

MAJOR

.

3

MINOR

.

1

PATCH

MAJOR

Breaking change — consumers MUST update

e.g. Removed a field, changed endpoint URL, renamed a method

MINOR

New backward-compatible functionality added

e.g. Added optional field, new endpoint, expanded enum

PATCH

Backward-compatible bug fix only

e.g. Fixed a calculation, corrected response data

Managing Breaking Changes — 3 Options



Lockstep Deployment

Deploy service + ALL consumers simultaneously. Simple but destroys independent deployability. Use only for same-team, one-off scenarios.

 Last resort

Coexist Incompatible Versions

Run v1 and v2 of the service simultaneously, routing traffic by version. Netflix uses this for legacy set-top boxes. Doubles operational burden.

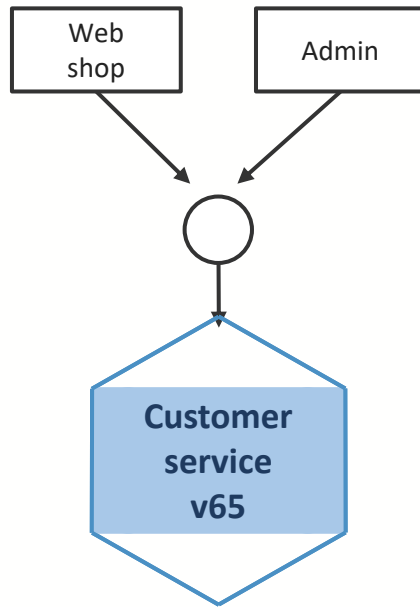
 Short-term only

Emulate the Old Interface

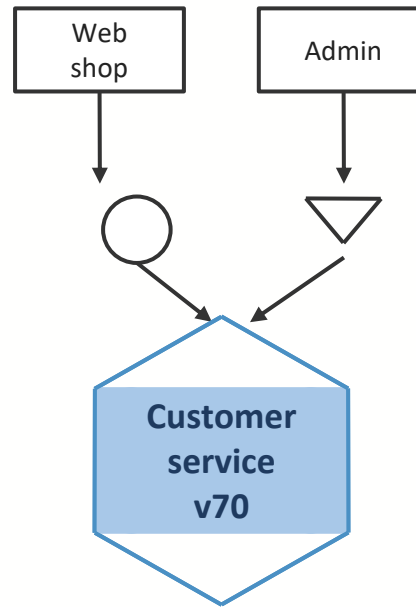
Deploy one service that exposes BOTH old and new interfaces. Consumers migrate at their own pace. Remove old interface once all consumers have upgraded.

 Recommended

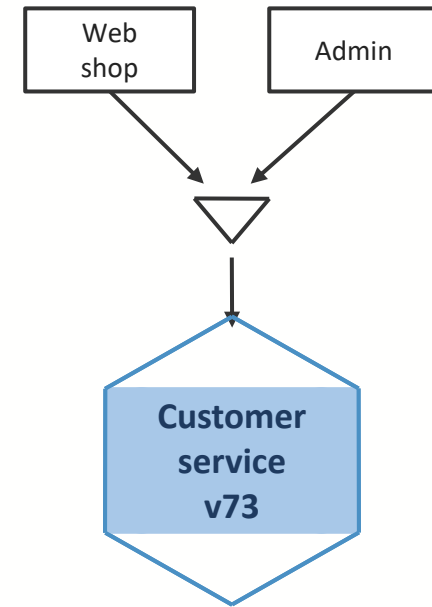
API Versioning Strategy



Single endpoint supported



*Expose old and new endpoints
from same microservice*



*Old endpoint removed once
consumers upgraded*

Versioning in APIs — URI vs Header

```
1 from fastapi import FastAPI, Header
2 from typing import Optional
3
4 app = FastAPI()
5
6 # — OPTION 1: Version in URI
7 @app.get("/v1/customers/{id}") # Old interface
8 async def get_customer_v1(id: str):
9     customer = await db.get(id)
10    # Returns old format (first_name, last_name separately)
11    return {"id": customer.id, "first_name": customer.first_name,
12           "last_name": customer.last_name, "email": customer.email}
13
14 @app.get("/v2/customers/{id}") # New interface
15 async def get_customer_v2(id: str):
16     customer = await db.get(id)
17     # Returns new format (full_name combined)
18     return {"id": customer.id, "full_name": customer.full_name,
19           "email": customer.email, "phone": customer.phone}
20
21 # — OPTION 2: Version in Accept header
22 @app.get("/customers/{id}")
23 async def get_customer_versioned(
24     id: str,
25     accept: Optional[str] = Header(default="application/vnd.api.v1+json")
26 ):
27     if "v2" in accept:
28         return await get_customer_v2(id)
29     return await get_customer_v1(id)
```

💡 URI versioning is most explicit; header versioning keeps URIs clean but is less obvious

DRY in Microservices



When Code Reuse Becomes a Liability

✓ SAFE TO SHARE

Logging libraries • Auth utilities • Telemetry helpers

These are internal concerns — invisible across service boundaries.

⚡ CLIENT LIBRARIES

Convenient — but risky when they contain business logic.

AWS model: SDKs are written by the community, not the API team — clients own the upgrade decision.

✗ DANGEROUS TO SHARE

Domain objects: Customer, Order, Product entities

One model change → all services must redeploy simultaneously.

★ THE RULE

Clients decide WHEN to upgrade.

Never force simultaneous upgrades across services.



Service Discovery

How microservices find each other at runtime

The Service Discovery Problem



- Problem: in a dynamic cloud environment, services start/stop/move constantly
- You can't hardcode IP addresses — they change on every container restart
- Service discovery = mechanism to register and find running service instances
- Two core responsibilities:
 1. Service registers itself when it starts up;
 2. Consumers look it up
- Goal: consumers don't need to know WHERE a service lives, just WHAT it's called

DNS — Simplest Service Discovery

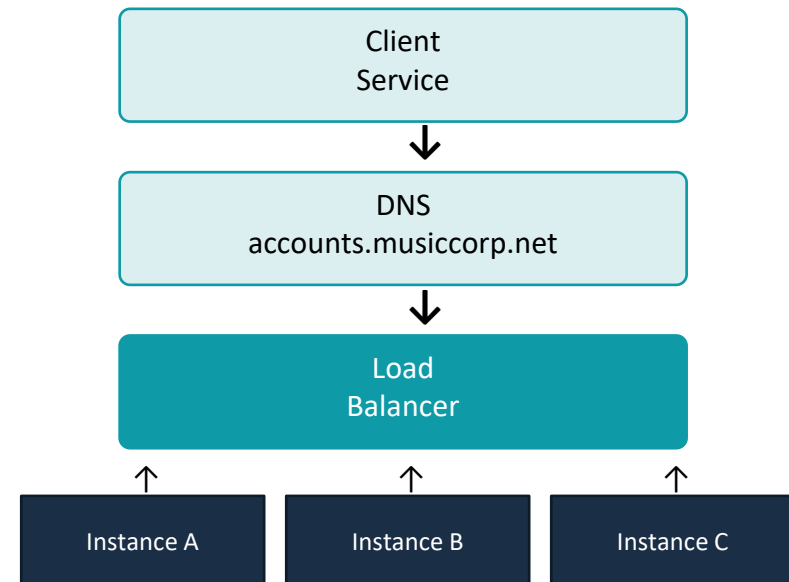
How It Works

- Each service gets a stable DNS name
- Convention: <service>-<env>.domain.net
- DNS entry → Load Balancer → Instances
- Works with any tech stack

Pros & Cons

- ✓ Zero new infrastructure needed
- ✓ Universal — any language/framework
- ⚠ TTL caching → stale IPs after changes
- ⚠ Not built for ephemeral containers

Request Flow



Consul — Dynamic Service Registry



HTTP API for Registration

Services register via REST on startup.
GET /v1/catalog/service/{name} to discover.

Automatic Health Checks

Consul polls /health on every instance.
Unhealthy instances are auto-deregistered instantly.

Built-in DNS Server

Serves SRV records — a drop-in replacement for DNS that updates dynamically.

Key/Value Config Store

consul-template updates config files live — services pick up changes without a restart.

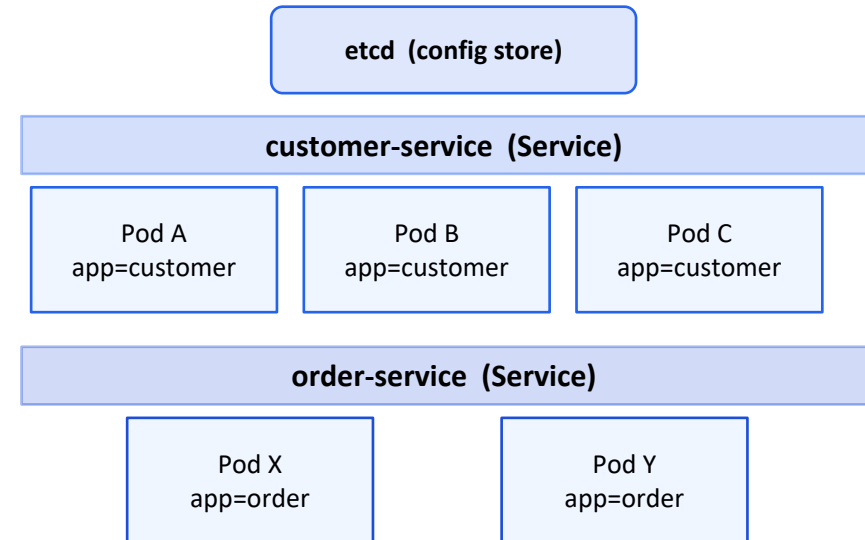
Kubernetes — Built-in Service Discovery

How Kubernetes Does It

- Each Service object gets a stable cluster-internal DNS name
- kube-proxy routes traffic to healthy Pods automatically
- Pods are labelled (e.g. app=customer); Services use selectors to find them
- etcd stores all cluster state & config

- ✓ Zero extra tooling — works out of the box
- ✓ Auto load-balancing + health checks included
- ⚠ Only works inside a Kubernetes cluster

Kubernetes Cluster





Service Meshes & API Gateways

Managing traffic at the infrastructure level

API Gateway vs Service Mesh



API GATEWAY

*Handles EXTERNAL traffic
(North–South)*

- Authentication & API keys
- Rate limiting & throttling
- Routing external requests inward
- Logging & analytics for consumers
- Developer portals



SERVICE MESH

*Handles INTERNAL traffic
(East–West)*

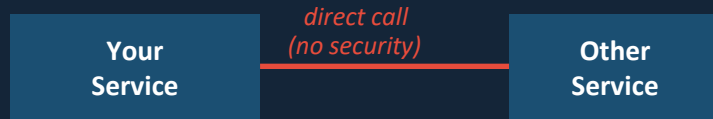
- mTLS between services (zero-trust)
- Service discovery & load balancing
- Circuit breaking & retries
- Distributed tracing
- Works across all languages

Sidecar Proxy Architecture



A separate process running alongside your service — your code never changes.

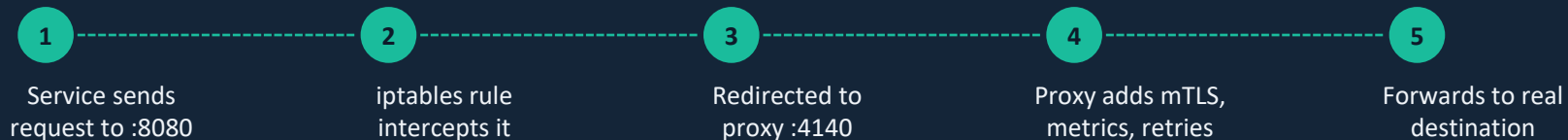
WITHOUT Sidecar



WITH Sidecar



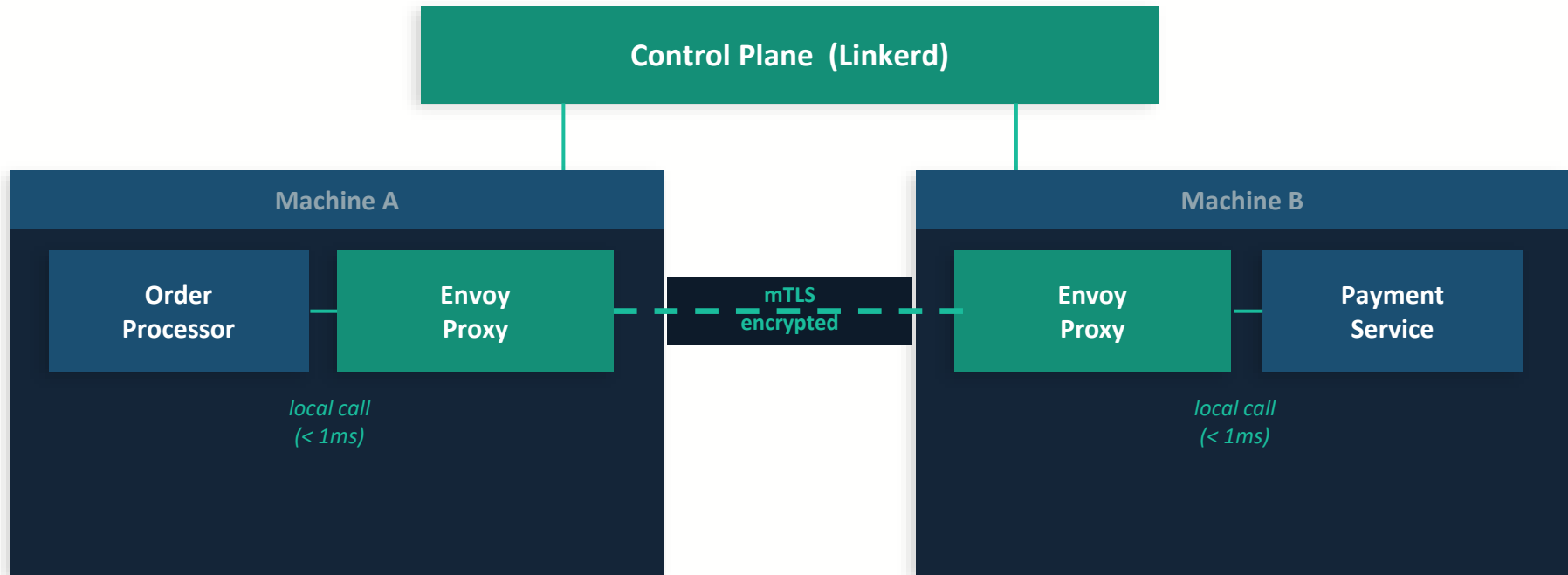
How traffic gets intercepted — iptables magic



Service Mesh: Sidecar Proxy Architecture



Each service gets a lightweight proxy running alongside it — your code never changes.



Key insight: Your service code is unchanged. The proxy handles encryption, retries, tracing, and load balancing automatically.

API Gateway: 4 Mistakes to Avoid



1. Call Aggregation in the Gateway

- ❌ Combining multiple service calls in the gateway creates a distributed monolith
- ✅ Use GraphQL or the BFF pattern instead

2. Protocol Rewriting (SOAP → REST)

- ❌ Rewriting protocols in the gateway makes it too smart — 'dumb pipes, smart endpoints'
- ✅ Each service should handle its own protocol. Keep the gateway thin

3. Routing ALL Inter-Service Traffic

- ❌ Sending east-west calls through the gateway adds unnecessary hops and latency
- ✅ Use a Service Mesh for internal calls; Gateway only for external entry points

4. Buying a Heavyweight Enterprise Gateway

- ❌ Expensive enterprise gateways are overkill if you only need routing or reverse proxy
- ✅ For Kubernetes: Ambassador or nginx are often enough — evaluate before buying

Decision Guide: What Should You Use?



Scenario	Recommendation	Why
External APIs (web or mobile clients)	API Gateway	Auth, rate limiting, routing to internal services
Internal service calls — small system (< 5 services)	Direct HTTP / gRPC	Mesh overhead not justified; keep it simple
Internal calls — large system (20+ services)	Service Mesh (Linkerd)	Consistent policy enforcement, mTLS, distributed tracing
Kubernetes-only deployment	K8s Service (built-in)	Kubernetes handles discovery & routing natively
Mixed infra (K8s + VMs + legacy systems)	Consul + Service Mesh	Works across heterogeneous platforms
Call aggregation for mobile / frontend	GraphQL or BFF	NOT API Gateway logic — aggregation belongs in a service

Microservice Communication

The 10 Commandments — Last Words

The 10 Commandments of Microservice Communication



1

Pick your communication style before picking technology

2

Always treat remote calls as what they are — remote, not local

3

Use explicit schemas — never let contracts be implicit

4

Apply the tolerant reader pattern in every consumer

5

Make expansion changes only; never remove without a plan

6

Set timeouts on every single remote call — always

7

Prefer emulating old interfaces over lockstep deployments

8

Keep your API gateway simple — no business logic

9

Use a service mesh for east-west; API gateway for north-south

10

Document your services

Textbooks

- Sam Newman, "Building Microservices: Designing Fine-Grained Systems," O'Reilly Media, 2021.



Thank You!

Questions?